

PROJECT DOCUMENTATION
ON
STUDENT MANAGEMENT SYSTEM
CERTIFICATE

This is to certify that **CHELLU MOUNIKA DURGA** , Bearing with roll no **246W1A6176**, of Artificial Intelligence And Machine Learning – D1, International school of technology and sciences for women, Rajahmundry. Has successfully completed the project on “Web Development” as part of the academic/project requirements under the guidance and supervision of **Ms.Sheik Janiya Jamgin**

Student Details

Student Name	CHELLU MOUNIKA DURGA
Roll Number	246W1A6176
College	International School of Technology & Sciences for Women
Branch & Section	AIML-D1
Under the Guidance of	Ms. Sheik Janiya Jamgin

Guide Signature: _____

Date: _____

TABLE OF CONTENTS / INDEX

S No	Project Title	Page number
1.	Calculator	3-11
2.	To-Do List	12-21
3.	Login Page	22-25
4.	Registration page	26-29

PROJECT 1:

1. INTRODUCTION

calculator:

A calculator program is a software application designed to accept numbers and mathematical operators from a user, process the computation, and output the correct result.

A Calculator Program is a simple computer program used to perform basic arithmetic operations. It allows the user to enter two numbers and select an operation such as addition, subtraction, multiplication, or division. The program processes the selected operation and displays the result.

2. OBJECTIVES

- To perform basic arithmetic operations such as addition, subtraction, multiplication, and division.
- To accept input from the user.
- To understand and apply arithmetic operators in programming.
- To use conditional statements or switch-case to select operations.
- To display the calculated result clearly.
- To develop basic problem-solving and programming logic skills.
- To create a simple and user-friendly calculator application.

3. TECHNOLOGIES USED

The project uses the following technologies:

- HTML : Used to create the basic structure and layout of the web pages.Used for headings, div and script linking.
- CSS : Used to design and format the user interface.
- JavaScript : Used for logic, perform arithmetic operations.
- Browser : Used to run and test the calculator application.
- VS Code : Used as the code editor.

4. SYSTEM REQUIREMENTS

Hardware Requirement:

- Computer/Laptop
- Minimum 4GB RAM
- Keyboard and mouse

Software Requirements:

- Windows/Linux/macOS
- Visual Studio code or any code editor
- Modern web browser such as chrome or edge

5. PROJECT DESCRIPTION

A calculator is an electronic or software-based device designed to perform mathematical calculations quickly and accurately. It is one of the most commonly used tools in education, business, engineering, science, and everyday activities. A calculator can perform basic arithmetic operations such as addition, subtraction, multiplication, and division. Depending on its type, it can also perform advanced operations such as percentages, square roots, powers, logarithmic functions, and trigonometric calculations.

The main purpose of developing a calculator is to make mathematical calculations simple, fast, and reliable. It reduces the time required for manual calculations and minimizes the possibility of human errors.

6. MODULES

Calculator Program Modules

- **Input Module** : Accepts numbers entered by the user.
- **Operation Selection Module** : Allows the user to select the required arithmetic operations.
- **Arithmetic Operations Module** : performs addition, subtraction, multiplication, and division.
- **Calculation Module** : processes the selected operation using java script logic.
- **Result Display Module** : Display the calculated result to the user
- **Error Handling Module** : Handles invalid inputs and prevents errors such as division by zero.

7.CODE:

CALCULATOR:

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Calculator</title>
<style>
  * {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
    font-family: Arial, sans-serif;
  }
  body {
    min-height: 100vh;
    display: flex;
    justify-content: center;
    align-items: center;
    background: linear-gradient(135deg, #667eea, #764ba2);
  }
  .calculator {
    width: 350px;
    padding: 20px;
    border-radius: 20px;
    background: #1e1e2f;
    box-shadow: 0 20px 50px rgba(0, 0, 0, 0.4);
  }
  .display {
    width: 100%;
    height: 90px;
    margin-bottom: 20px;
    padding: 15px;
    border: none;
    outline: none;
    border-radius: 12px;
    background: #11111c;
    color: white;
    font-size: 38px;
```

```

        text-align: right;
    }
.buttons {
    display: grid;
    grid-template-columns: repeat(4, 1fr);
    gap: 12px;
}
button {
    height: 65px;
    border: none;
    border-radius: 12px;
    font-size: 22px;
    font-weight: bold;
    cursor: pointer;
    color: white;
    background: #33334d;
    transition: 0.15s;
}
button:hover {
    background: #454566;
}
button:active {
    transform: scale(0.94);
}
.operator {
    background: #ff9500;
}
.operator:hover {
    background: #e68600;
}
.function {
    background: #666680;
}
.equals {
    background: #28a745;
}
.equals:hover {
    background: #218838;
}
.zero {
    grid-column: span 2;
}
@media (max-width: 400px) {

```

```

        .calculator {
            width: 95%;
        }
        button {
            height: 60px;
        }
    }
</style>
</head>
<body>
    <div class="calculator">
        <input
            type="text"
            id="display"
            class="display"
            value="0"
            readonly
        >
        <div class="buttons">
            <button class="function" onclick="clearDisplay()">AC</button>
            <button class="function" onclick="deleteLast()">DEL</button>
            <button class="function" onclick="percentage()">%</button>
            <button class="operator" onclick="addOperator('/')">÷</button>
            <button onclick="addNumber('7')">7</button>
            <button onclick="addNumber('8')">8</button>
            <button onclick="addNumber('9')">9</button>
            <button class="operator" onclick="addOperator('*')">×</button>
            <button onclick="addNumber('4')">4</button>
            <button onclick="addNumber('5')">5</button>
            <button onclick="addNumber('6')">6</button>
            <button class="operator" onclick="addOperator('-)">-</button>
            <button onclick="addNumber('1')">1</button>
            <button onclick="addNumber('2')">2</button>
            <button onclick="addNumber('3')">3</button>
            <button class="operator" onclick="addOperator('+)">+</button>
            <button class="zero" onclick="addNumber('0')">0</button>
            <button onclick="addDecimal()">.</button>
            <button class="equals" onclick="calculate()">=</button>
        </div>
    </div>
</body>
<script>
    const display = document.getElementById("display");
    let currentInput = "0";

```

```

let expression = "";
function addNumber(number) {
  if (currentInput === "0" || currentInput === "Error") {
    currentInput = number;
  } else {
    currentInput += number;
  }
  updateDisplay();
}
function addDecimal() {
  if (currentInput === "Error") {
    currentInput = "0";
  }
  if (!currentInput.includes(".")) {
    currentInput += ".";
  }
  updateDisplay();
}
function addOperator(operator) {
  if (currentInput === "Error") {
    currentInput = "0";
  }
  expression += currentInput + " " + operator + " ";
  currentInput = "0";
  updateDisplay();
}
function calculate() {
  try {
    let fullExpression = expression + currentInput;
    if (!/^[0-9+\-*/. ]+$/.test(fullExpression)) {
      throw new Error();
    }
    let result = Function(
      "use strict"; return (' + fullExpression + ')
    )();
    if (!Number.isFinite(result)) {
      throw new Error();
    }
    currentInput = String(
      Math.round((result + Number.EPSILON) * 100000000) / 100000000
    );
    expression = "";
    updateDisplay();
  }
}

```

```

        } catch {
            currentInput = "Error";
            expression = "";
            updateDisplay();
        }
    }

    function clearDisplay() {
        currentInput = "0";
        expression = "";
        updateDisplay();
    }

    function deleteLast() {
        if (currentInput === "Error") {
            clearDisplay();
            return;
        }
        if (currentInput.length > 1) {
            currentInput = currentInput.slice(0, -1);
        } else {
            currentInput = "0";
        }
        updateDisplay();
    }

    function percentage() {
        if (currentInput === "Error") {
            clearDisplay();
            return;
        }
        currentInput = String(Number(currentInput) / 100);
        updateDisplay();
    }

    function updateDisplay() {
        display.value = currentInput;
    }

    document.addEventListener("keydown", function(event) {
        const key = event.key;
        if (key >= "0" && key <= "9") {
            addNumber(key);
        }
        else if (key === ".") {
            addDecimal();
        }
        else if (key === "+" || key === "-" ||

```

```

        key === "*" || key === "/" ) {
    addOperator(key);
}
else if (key === "Enter" || key === "=") {
    event.preventDefault();
    calculate();
}
else if (key === "Backspace") {
    deleteLast();
}
else if (key === "Escape") {
    clearDisplay();
}
else if (key === "%") {
    percentage();
}
});
</script>
</body>
</html>

```

Output:



10.Conclusion:

The development of this calculator program successfully fulfills the objective of creating a reliable, user-friendly tool for mathematical computations. By breaking the application down into modular components—separating the math logic, user interface, and error handling—the project achieves high code maintainability and scalability.

PROJECT -2:

1. INTRODUCTION

TO-DO LIST:

A To-Do List is a simple and effective tool used to organize and manage daily tasks. It helps users keep track of activities that need to be completed, making it easier to plan work and improve productivity. A To-Do List application allows users to add, view, update, and delete tasks as needed. By maintaining a list of pending and completed tasks, users can stay organized, manage their time efficiently, and ensure that important activities are not forgotten. This project demonstrates the basic concepts of task management and can be developed using technologies such as HTML, CSS, JavaScript, Python, or database systems.

2. OBJECTIVES

A To-Do List is a task management application that helps users organize and track their daily activities. It allows users to create, update, and delete tasks based on their needs. The system helps users prioritize work, monitor task completion, and improve productivity. By maintaining a list of pending and completed tasks, the application ensures better time management and reduces the risk of forgetting important activities.

3. TECHNOLOGIES USED

The project uses the following technologies:

- HTML : Used to create the basic structure and layout of the web pages.Used for headings, divs and script linking.
- CSS : Used to design and format the user interface.
- JavaScript : Used for logic, DOM manipulation and event handling.
- Browser :Used to run and test the application.
- VS Code : Used as the code editor.

4. SYSTEM REQUIREMENTS

Hardware Requirement:

- Computer/Laptop
- Minimum 4GB RAM
- Keyboard and mouse

Software Requirements:

- Windows/Linux/macOS
- Visual Studio code or any code editor
- Modern web browser such as chrome or edge

5. PROJECT DESCRIPTION

A To-Do List is a task management application that helps users organize and track their daily activities. It allows users to create, update, and delete tasks based on their needs. The system helps users prioritize work, monitor task completion, and improve productivity. By maintaining a list of pending and completed tasks, the application ensures better time management and reduces the risk of forgetting important activities.

6. MODULES

To-Do List Modules

- **User Login Module** : User authentication and account management.
- **Task Creation Module** : Add new tasks to the list.
- **Task Update Module** : Edit existing task details.
- **Task Deletion Module** : Remove completed or unwanted tasks

7. CODE

TO-DO LIST:

HTML CODE:

index.html

```
<!DOCTYPE html>
<html>
<head>
  <title>To-Do List</title>
  <style>
    body {
      font-family: Arial;
      padding: 30px;
    }
    input, select, button {
      padding: 8px;
      margin: 5px;
    }
    li {
      margin: 10px;
      padding: 10px;
      border: 1px solid gray;
      width: 500px;
    }
    .completed {
      text-decoration: line-through;
    }
    .high {
      Colour : red;
    }
    .medium {
      colour: orange;
    }
    .low {
      color: green;
    }
  </style>
</head>
```

```

<body>
  <h1> To-Do List</h1>
  <!-- Task Input -->
  <input
    type="text"
    id="
taskInput"
    placeholder="Enter task"
  >
  <!-- Date -->
  <input
    type="date"
    id="taskDate"
  >
  <!-- Priority -->
  <select id="priority">
    <option value="High">High</option>
    <option value="Medium">Medium</option>
    <option value="Low">Low</option>
  </select>
  <button id="addBtn">
    Add Task
  </button>

  <button id="clearBtn">
    Clear All
  </button>
  <br><br>
  <!-- Search -->
  <input
    type="text"
    id="searchInput"
    placeholder=" Search task"
  >
  <!-- Filter -->
  <select id="filter">
    <option value="all">All</option>
    <option value="pending">Pending</option>
    <option value="completed">Completed</option>
  </select>

```

```

<!-- Counter -->
<h3 id="counter">
  Total: 0 | Completed: 0 | Pending: 0
</h3>
<!-- Task List -->
<ol id="taskList"></ol>
<script>
  let taskInput = document.getElementById("taskInput");
  let taskDate = document.getElementById("taskDate");
  let priority = document.getElementById("priority");
  let addBtn = document.getElementById("addBtn");
  let clearBtn = document.getElementById("clearBtn");
  let searchInput = document.getElementById("searchInput");
  let filter = document.getElementById("filter");
  let taskList = document.getElementById("taskList");
  let counter = document.getElementById("counter");
  let tasks = [];
  let editIndex = -1;
  addBtn.addEventListener("click", function () {
    let task = taskInput.value.trim();
    if (task === "") {
      alert("Please enter a task!");
      return;
    }
    if (editIndex !== -1) {
      tasks[editIndex].name = task;
      tasks[editIndex].date = taskDate.value;
      tasks[editIndex].priority = priority.value;
      editIndex = -1;
      addBtn.textContent = "Add Task";
    }
    else {
      let newTask = {
        name: task,
        date: taskDate.value,
        priority: priority.value,
        completed: false;  };
      tasks.push(newTask);
    }
    taskInput.value = "";
  });

```

```

    taskDate.value = "";
    displayTasks();
});
function displayTasks() {
    taskList.innerHTML = "";
    let searchText =
        searchInput.value.toLowerCase();
    let selectedFilter =
        filter.value;
    let displayNumber = 1;
    tasks.forEach(function (task, index) {
        if (
            !task.name
                .toLowerCase()
                .includes(searchText)
        ) {
            return;
        }
        if (
            selectedFilter === "pending"
            && task.completed === true
        ) {
            return;
        }
        if (
            selectedFilter === "completed"
            && task.completed === false
        ) {
            return;
        }
        let li = document.createElement("li");
        let span = document.createElement("span")
        span.textContent =
            task.name + " ";
        if (task.completed) {
            span.classList.add("completed");}
        let priorityText =
            document.createElement("span");
        priorityText.textContent =
            " [" + task.priority + "] ";

```

```

priorityText.classList.add(
    task.priority.toLowerCase()
);
let dateText =
    document.createElement("span");
if (task.date !== "") {
    dateText.textContent =
        "  " + task.date;
}
let completeBtn =
    document.createElement("button");
completeBtn.textContent =
    "Complete";
completeBtn.addEventListener(
    "click",
    function () {
        task.completed =
            !task.completed;
        displayTasks();
    });
let editBtn =
    document.createElement("button");
editBtn.textContent =
    "Edit";
editBtn.addEventListener(
    "click",
    function () {
        taskInput.value =
            task.name;
        taskDate.value =
            task.date;
        priority.value =
            task.priority;
        editIndex = index;
        addBtn.textContent =
            "Update Task";
    });
let deleteBtn =
    document.createElement("button");
deleteBtn.textContent =

```

```

        "Delete";
deleteBtn.addEventListener(
    "click",
    function () {
        tasks.splice(index, 1);
        displayTasks();
    } );
li.appendChild(span);
li.appendChild(priorityText);
li.appendChild(dateText);
li.appendChild(completeBtn);
li.appendChild(editBtn);
li.appendChild(deleteBtn);
taskList.appendChild(li);
displayNumber++;
});
updateCounter();
}
function updateCounter() {
    let total =
        tasks.length;
    let completed =
        tasks.filter(function (task) {
            return task.completed === true;
        }).length;
    let pending =
        total - completed;
counter.textContent = "Total: " + total +
    " | Completed: " + completed +
    " | Pending: " + pending;
}
clearBtn.addEventListener(
    "click",
    function () {
        tasks = [];
        displayTasks();
    } );
searchInput.addEventListener

    "input",
    function () {

```

```
        displayTasks();
    } );
filter.addEventListener(
    "change",
    function () {
        displayTasks();
    });
taskInput.addEventListener(
    "keydown",
    function (event) {
        if (event.key === "Enter") {
            addBtn.click();
        }
    });
</script>
</body>
</html>
```

OUTPUT:



To-Do List

Enter task

dd-mm-yyyy



High

▼

Add Task

Clear All



Search task

All

▼

Total: 2 | Completed: 0 | Pending: 2

1.

READING [Medium]  2026-09-12

Complete

Edit

Delete

2.

WRITING [High]  2026-09-08

Complete

Edit

Delete

10. CONCLUSION

The To-Do List helps users organize and manage their daily tasks effectively. It allows users to add, edit, delete, and complete tasks, helping improve productivity and time management

PROJECT -3:

1. INTRODUCTION

LOGIN PAGE:

A Login Page is a web page that allows registered users to securely access their accounts. It usually requires a **username/email and password** for authentication. The login system verifies the entered details and provides access to authorized users. It helps protect user information and prevents unauthorized access

2. OBJECTIVES

Login Page

- To provide secure user authentication.
- To verify username and password.
- To prevent unauthorized access.
- To protect user account information.
- To provide easy access to registered users

3. TECHNOLOGIES USED

The project uses the following technologies:

- **HTML** : Used to create the basic structure and layout of the web pages.Used for headings, divs and script linking.
- **CSS** : Used to design and format the user interface.
- **JavaScript** : Used for logic, DOM manipulation and event handling.
- **Browser** : Used to run and test the application.
- **VS Code** : Used as the code editor.

4. SYSTEM REQUIREMENTS

Hardware Requirement:

- Computer/Laptop
- Minimum 4GB RAM
- Keyboard and mouse

Software Requirements:

- Windows/Linux/macOS
- Visual Studio code or any code editor
- Modern web browser such as chrome or edge

5. PROJECT DESCRIPTION

Login Page

Login Page is a web-based application used to authenticate registered users. It allows users to enter their username or email and password to access their accounts. The system verifies the entered details and grants access only to authorized users. It also provides error messages for invalid login details and helps maintain account security.

6. MODULES

Login Page-Modules

- **User Input Module** : Accepts username/email and password.
- **Validation Module** : Checks whether the entered details are valid.
- **Authentication Module** : Verifies user credentials.
- **Error Handling Module** : Displays messages for incorrect details.
- **Password Module** : Handles password visibility and security.
- **Access Module** : Provides access to authorized users.

7. CODE

LOGIN PAGE:

index.html

```
<!Doctype.html>
<html>
<head>
  <title>login Page</title>
</head>
<style>
  body{
    color: rgb(120, 42, 165);
  }
  h1 {
    color:brown;
  }
  label{
    color: rgb(50, 49, 58);
  }
  input{
    color: rgb(165, 42, 42);
  }
  button{
    background-color: cadetblue;
  }
  p{
    color: rgb(255, 0, 191);
  }
</style>
<body>
  <h1>login</h1>
  <label>Username or emailid:</label>
  <input type="text" id="username or emailid">
  <br><br>
  <label>Password:</label>
  <input type="password" id="password">
  <br><br>
  <button id="loginBtn">Login</button>
  <p id="message"></p>
  <script>
    let username=document.getElementById("username or emailid");
    let password=document.getElementById("password");
    let loginBtn=document.getElementById("loginBtn");
    let message=document.getElementById("message");
```

```

loginBtn.addEventListener("click",function() {
    let user=username.value;
    let pass=password.value;
    if(user=="Madhu" || "madhushalini@gmail.com" && pass=="1234")
    {
        message.innerHTML="login Succesfull";
    } else {
        message.innerHTML="Invalid username or password";
    }
});
</script>
</body>
</html>

```

OUTPUT:

Login

Username:

Password:

Login Successfull

Login

Username:

Password:

Invalid username or password

10. CONCLUSION

The Login Page provides a secure way for registered users to access their accounts. It verifies user credentials, prevents unauthorized access, and provides a simple interface for authentication.

PROJECT - 4 :

1. INTRODUCTION

REGISTRATION PAGE:

A Registration Page is a simple and useful web application that allows users to create an account by entering their personal information such as name, email, phone number, username, and password. The registration page collects and validates the user's information before submitting the form. It helps ensure that the entered details are in the correct format and provides error messages when invalid information is entered. This project demonstrates the basic concepts of web development using HTML, CSS, and JavaScript.

2. OBJECTIVES

The main objective of the Registration Page is to provide a simple and user-friendly interface for creating a new user account. The system allows users to enter their personal details and password securely. JavaScript is used to validate the entered information and display appropriate error messages. The project helps in understanding form creation, styling, input validation, DOM manipulation, and event handling in web development.

3. TECHNOLOGIES USED

The project uses the following technologies:

- HTML : Used to create the basic structure and layout of the registration page.
- CSS : Used to design and format the registration form and improve the user interface.
- JavaScript : Used for form validation, logic, DOM manipulation, and event handling.
- Browser : Used to run and test the registration page.
- VS Code : Used as the code editor for writing and modifying the code.

4. SYSTEM REQUIREMENTS

Hardware Requirement:

- Computer/Laptop
- Minimum 4GB RAM
- Keyboard and mouse

Software Requirements:

- Windows/Linux/macOS
- Visual Studio Code or any code editor
- Modern web browser such as Chrome or Edge

5. PROJECT DESCRIPTION

A Registration Page is a web-based application that allows users to create a new account by entering their required details. The page contains fields such as full name, email address, phone number, password, and confirm password.

The application validates the information entered by the user before submitting the form. It checks whether all required fields are filled and whether the password and confirm password match. If incorrect information is entered, appropriate error messages are displayed. When all the details are valid, a registration success message is displayed.

6. MODULES

Registration Page Modules

- User Information Module : Collects the user's name, email, and phone number.
- Password Module : Allows the user to enter and confirm their password.
- Form Validation Module : Validates the information entered by the user.
- Error Message Module : Displays error messages when incorrect or incomplete information is entered.
- Registration Module : Processes the registration form after successful validation.

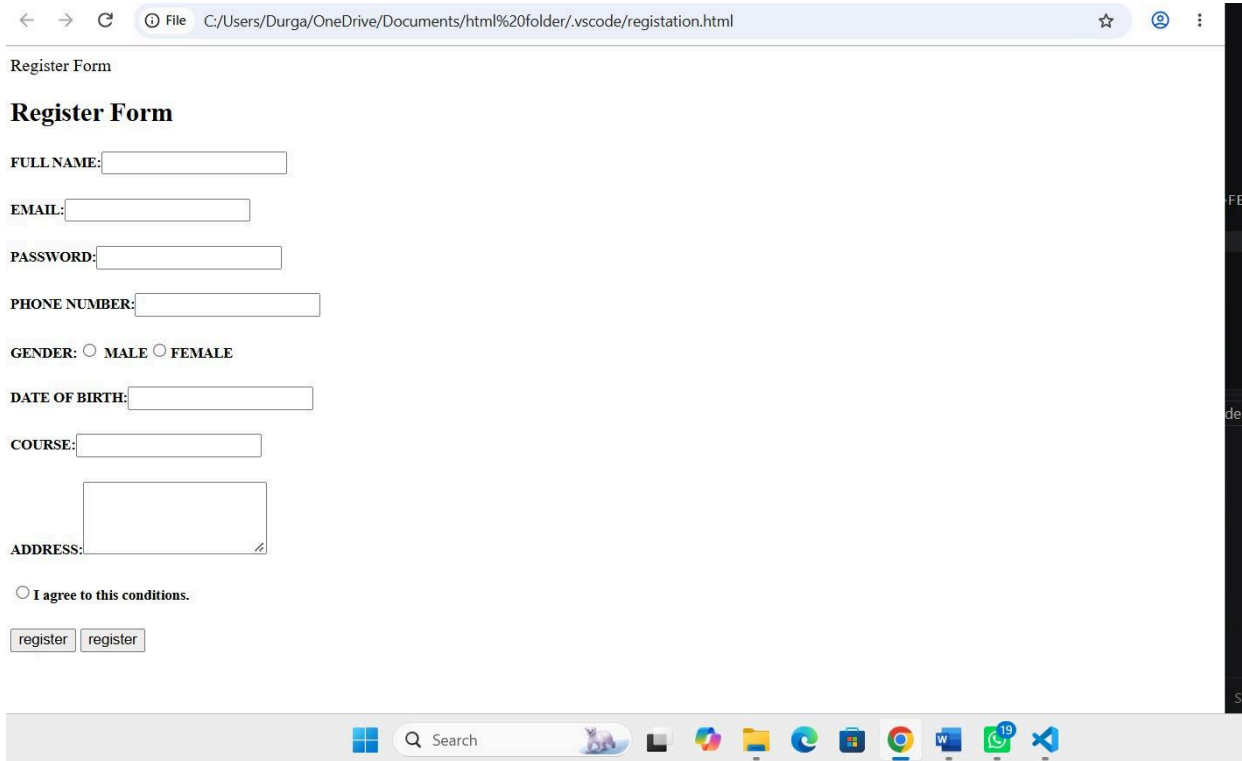
7. CODE

REGISTRATION PAGE:

index.html

```
<html lang="en">
<head>
  <titl>Register Form</titl></title>
</head>
<body>
  <div class="login">
    <h2>Register Form</h2>
    <form action="">
      <h5>FULL NAME:<input type="text" id="user"></h5>
      <h5> EMAIL:<input type="text" id="user"></h5>
      <h5> PASSWORD:<input type="text" id="user"></h5>
      <h5> PHONE NUMBER:<input type="text" id="user"></h5>
      <h5>GENDER:<input type="radio" name="gender" value="male"> MALE<input
type="radio" name="gender" value="female">FEMALE</h5>
      <h5> DATE OF BIRTH:<input type="text" id="user"></h5>
      <h5> COURSE:<input type="text" id="user"></h5>
      <h5>ADDRESS:<textarea name="address" rows="4" columns="35"></textarea></h5>
      <h5><input type="radio">I agree to this conditions. </h5>
    </form>
  </div>
  <button onclick="clিকেবন্ট(name.value)">register</button>
  <button onclick="clিকেবন্ট2(name.value)">register</button>
  <script>
    function clিকেবন্ট2(name) {
      alert("restart your registration");
    }
  </script>
</body>
</html>
```

OUTPUT:



The screenshot shows a web browser window with the address bar displaying the file path: C:/Users/Durga/OneDrive/Documents/html%20folder/vscode/registration.html. The page title is "Register Form". The form itself is titled "Register Form" and contains the following fields and elements:

- FULL NAME:** A text input field.
- EMAIL:** A text input field.
- PASSWORD:** A text input field.
- PHONE NUMBER:** A text input field.
- GENDER:** Radio buttons for ☐ MALE and ☐ FEMALE.
- DATE OF BIRTH:** A text input field.
- COURSE:** A text input field.
- ADDRESS:** A larger text input field.
- ☐ I agree to this conditions.
- Two "register" buttons.

The browser's taskbar at the bottom shows various application icons including Windows, Search, and several web and utility applications.

8. CONCLUSION

The Registration Page project provides a simple and user-friendly interface for creating a new user account. It collects important user details such as name, email, phone number, and password. HTML is used to create the structure, CSS is used for designing the page, and JavaScript is used for form validation and functionality. This project helps in understanding basic web development concepts such as forms, input validation, DOM manipulation, and event handling.

References

1. HTML, CSS and JavaScript documentation.
2. MDN Web Docs.
3. W3Schools learning resources.
4. Class notes and project guidance.
5. Other resources used during project development.